

21 — System Design with PostgreSQL

Overview

This folder contains eight end-to-end system design documents with PostgreSQL at the core. Each document follows a consistent structure: requirements → capacity estimation → schema design → indexing → scaling → tradeoffs → interview guidance. Together they form a complete reference for database-focused system design interviews.

Files

File	System	Key PostgreSQL Features
01_url_shortener.md	URL Shortener	Sequences, base62 encoding, caching, click analytics
02_ecommerce_system_design.md	E-Commerce	Inventory reservation, saga pattern, FTS, partitioning
03_banking_system_design.md	Banking	ACID, SERIALIZABLE, double-entry ledger, stored procedures
04_ride_sharing_design.md	Ride-Sharing	PostGIS, UNLOGGED tables, geospatial matching
05_social_network_design.md	Social Network	Feed fan-out, pg_trgm search, partitioning, denormalization
06_chat_application_design.md	Chat App	LISTEN/NOTIFY, cursor pagination, SKIP LOCKED, presence
07_analytics_platform_design.md	Analytics	BRIN index, materialized views, partitioning, COPY ingest
08_system_design_framework.md	Interview Framework	Six-step process, checklists, anti-patterns, key numbers

The Six-Step Framework (Quick Reference)

1. CLARIFY	→ Scale, features, constraints, compliance
2. ESTIMATE	→ QPS, storage/year, read:write ratio
3. SCHEMA	→ Tables, constraints, FKs, indexes
4. INDEXES	→ Type selection, composite order, partial indexes
5. SCALING	→ Ladder: pool → replica → cache → partition → shard
6. TRADEOFFS	→ Name what you gain AND what you give up

Key Design Decisions Summary

When to Use Which Index Type

Workload	Index Type
----------	------------

Equality + range queries	B-Tree composite
Full-text search	GIN (tsvector)
JSONB containment	GIN
Geospatial queries	GiST (PostGIS)
No-overlap constraint	GiST + EXCLUDE
Append-only time-series	BRIN
Sparse filter	Partial B-Tree
Sub-string search	GIN (pg_trgm)

Isolation Level Selection

Operation	Use	Reason
Fund transfers	SERIALIZABLE	Prevents write skew / double-spend
Balance read	READ COMMITTED	Current value, no phantom concern
Statement generation	REPEATABLE READ	Consistent multi-row snapshot
Analytics queries	READ COMMITTED	Speed; slight lag acceptable
Batch jobs	REPEATABLE READ	Consistent view across job

Scaling Decision Points

Symptom	Solution
Connection overload	PgBouncer (before any other change)
Read QPS > 10K	Read replica
Hot rows / repeated reads	Redis cache
Table > 100GB with time filter	Range partitioning
Write QPS > 50K	Horizontal sharding
Complex search on > 50M rows	Elasticsearch
Analytical queries on > 1TB	Columnar store (ClickHouse/BigQuery)

Schema Patterns Quick Reference

```
-- Soft delete
WHERE deleted_at IS NULL

-- Idempotent insert
INSERT ... ON CONFLICT (key) DO NOTHING
```

```

-- Idempotent update
INSERT ... ON CONFLICT (key) DO UPDATE SET col = EXCLUDED.col

-- Job queue (parallel workers)
SELECT ... FOR UPDATE SKIP LOCKED LIMIT $batch_size

-- Fan-out insert
INSERT INTO user_feeds SELECT follower_id, $post_id FROM follows WHERE followee_id =
$author

-- Optimistic lock
UPDATE table SET ... WHERE id = $1 AND version = $expected_version
-- (check rowcount; if 0, someone else updated first)

-- No-overlap constraint (booking systems)
EXCLUDE USING GIST (
    resource_id WITH =,
    daterange(start_date, end_date, '[') WITH &&
)

-- Generated computed column
quantity_available INT GENERATED ALWAYS AS (on_hand - reserved) STORED

```

Capacity Estimation Cheat Sheet

```

1M events/day  = 12 events/second  = ~500MB/day (at 500B/row)
1B events/day  = 11,600 events/sec  = ~500GB/day
100M users    × 1KB/user            = 100GB
1M orders/day × 300B/order           = ~100GB/year

```

PostgreSQL can handle:

- ~50K simple reads/second (indexed)
- ~10K simple writes/second
- ~500K rows/second via COPY
- ~10K concurrent connections (via PgBouncer)

Redis can handle:

- ~1M GET/SET operations/second per node
- Sub-millisecond latency

Rule: If a single item is read > 100x/second, put it in Redis.

Rule: If a table will exceed 100GB with time-based queries, partition it.

Rule: If write QPS exceeds 50K, consider sharding.

Prerequisite Knowledge

Before working through these design documents, ensure you understand:

- **Chapter 4:** Transactions, ACID, BEGIN/COMMIT/ROLLBACK

- **Chapter 5:** Table partitioning (RANGE, LIST, HASH)
- **Chapter 6:** Isolation levels (READ COMMITTED, REPEATABLE READ, SERIALIZABLE)
- **Chapter 7:** JSONB queries and GIN indexes
- **Chapter 8:** Row-Level Security
- **Chapter 9:** Full-text search (tsvector, tsquery, GIN)
- **Chapter 10:** LISTEN/NOTIFY
- **Chapter 11:** PostGIS extension (geography, geometry, GiST)
- **Chapter 12:** Connection pooling and performance tuning

Related Folder

After completing this folder, the architecture case studies in `18_Architecture_Case_Studies/` provide deeper schema detail for the same systems:

This folder	Case study equivalent
<code>02_ecommerce_system_design.md</code>	<code>18_.../01_amazon_marketplace_db.md</code>
<code>03_banking_system_design.md</code>	<code>18_.../04_banking_platform.md</code>
<code>04_ride_sharing_design.md</code>	<code>18_.../03_uber_ridesharing.md</code>
<code>06_chat_application_design.md</code>	<code>18_.../06_messaging_system.md</code>
(no equivalent)	<code>18_.../02_netflix_streaming_platform.md</code>
(no equivalent)	<code>18_.../05_saas_product.md</code>

How to Study

1. **Read `08_system_design_framework.md` first** — internalize the six-step process
2. **Work through one system per day** — read it, then try to reproduce the design from memory
3. **Practice estimation** — given any system, compute QPS and storage in under 5 minutes
4. **Know the anti-patterns** — being able to name what *not* to do is as valuable as knowing what to do
5. **Time yourself** — aim to present any design in 20 minutes with confidence